# TP1 - Relacionamento 1:N

Iniciar tarefa

- Vencimento Quarta-feira por 23:59
- Pontos 5
- Enviando um URL de site ou uma gravação de mídia
- Disponível até 15 abr em 23:59

Ao longo do semestre, implementaremos um sistema para **gestão de inscrições em cursos livres**, em que os alunos da PUC Minas podem ofertar seus próprios cursos ou se inscrever em cursos de outros alunos. Cada usuário, portanto, poderá cadastrar quantos cursos quiser. Esses cursos ficarão abertos para inscrição por outros usuários, que poderão se inscrever em quantos cursos quiserem.

Neste primeiro trabalho prático, criaremos os cursos, mas deixaremos a parte das inscrições para o segundo trabalho prático. No entanto, como os cursos serão ofertados por usuários específicos, precisaremos também criar os próprios usuários.

## O ACESSO AO SISTEMA

Para acessar o sistema, um usuário deverá estar cadastrado. O acesso será feito mediante email e senha. Caso o usuário não esteja cadastrado, deverá haver uma opção que permita o seu próprio cadastro. Assim, um primeira tela poderia ser algo assim:

```
EntrePares 1.0
--------------

[A] Login
[B] Novo usuário

[S] Sair

Opção: _
```

## A ENTIDADE USUÁRIO

Nossa entidade usuário precisará contar com pelo menos os seguintes atributos:

- Nome
- E-mail
- HashSenha
- PerguntaSecreta

- HashRespostaSecreta

Nós não armazenamos a senha de um usuário, mas o código hash retornado por ela. Como temos uma limitação de recursos a serem usados em uma interface textual, os atributos PerguntaSecreta e RespostaSecreta deverão ser usados para recuperação da senha. Lembrem-se de que toda entidade precisa de um identificador exclusivo (ID) que, como vimos nas aulas, será um número inteiro positivo sequencial ($\mathbb{N}^* = \{1, 2, 3, \ldots\}$). Notem que um usuário se identificará por meio do seu email e não do seu ID. A diferença é que o email pode ser alterado, o ID não.

## A ENTIDADE CURSO

Também precisaremos cadastrar os cursos.

Um curso pertencerá a apenas um usuário, mas um usuário poderá ter vários cursos. É por isso que dizemos que o relacionamento é de 1 para N (ou 1:N):

- 1 usuário tem N cursos
- 1 curso pertence a 1 único usuário

Cada curso deve ter, pelo menos, os seguintes atributos (além do ID):

- Nome
- Data de início do curso
- Descrição detalhada (programa, dias, locais, ...)
- Código compartilhável
- Estado

Aqui, o atributo código compartilhável é algo que merece uma atenção especial. Usaremos um código alfanumérico de **10 caracteres** seguindo o padrão da biblioteca **NanoID (https://github.com/ai/nanoid#readme)** e que será gerado automaticamente pelo sistema. Quando alguém quiser mostrar um curso a seus amigos (ou divulgar na Internet), deverá usar esse código. Observem que ele não se confunde com o ID do curso. Finalmente, vocês também precisarão de uma chave estrangeira nessa entidade (o ID do usuário).

O atributo de estado do curso indicará qual o estado atual do curso, podendo ser:

- 0 - Curso ativo e recebendo inscrições
- 1 - Curso ativo, mas sem novas inscrições (geralmente por já ter iniciado)
- 2 - Curso realizado e concluído
- 3 - Curso cancelado

Minha sugestão é de que vocês façam uma organização das operações desta forma:

```
EntrePares 1.0
--------------
> Início > Meus cursos

CURSOS
[1] Finanças pessoais - 10/02/2026
[2] Javascript para iniciantes - 15/04/2026
[3] Descubra o Python - 20/05/2026

[A] Novo curso
[R] Retornar ao menu anterior

Opção: _
```

Notem que, abaixo do título, estamos usando um *breadcrumb* para facilitar a percepção do usuário de onde ele está no sistema. Ao entrar no menu Meus Cursos, o usuário deveria ver todos os cursos que ele já criou.

Para visualizar os dados (ou editar) um curso, o usuário deve selecionar o curso desejado por meio do número no menu. Esse menu é construído a partir dos cursos cadastrados pelo usuário ativo. Observem que o número do curso no menu não corresponde ao ID da entidade, mas é um número sequencial a partir da **ordem alfabética dos cursos**. Para isso, vocês precisaram de um índice indireto baseado no nome do curso (usem uma árvore B+) que retorne todos os cursos do usuário ativo. A opção N deve permitir o cadastro de um novo curso.

Após a seleção do curso, vocês podem apresentar uma interface como esta:

```
EntrePares 1.0
--------------
> Início > Meus Cursos > Python básico

CÓDIGO........: tdfd9as8bp
NOME..........: Descubra o Python
DESCRIÇÃO.....: Este curso intensivo de 5 dias foi projetado para levar iniciantes do zero ao desenvolvimento de scripts f
uncionais em Python, focando em raciocínio lógico, sintaxe essencial e automação prática. O aluno aprenderá as estrut
uras básicas do Python; o controle de fluxo e lógica; as coleções e os laços de repetição; as funções e modularização; e
a manipulação de arquivos.
DATA DE INÍCIO: 20/05/2026

Este curso está aberto para inscrições!

[A] Gerenciar inscritos no curso
[B] Corrigir dados do curso
[C] Encerrar inscrições
[D] Concluir curso
[E] Cancelar curso

[R] Retornar ao menu anterior

Opção: _
```

Neste TP1, ainda não implementaremos a opção 1 desse menu. Isso ficará para o TP2. As três ultimas opções (C, D e E) estão relacionadas ao estado do curso. Ao encerrar inscrições, não será possível ninguém mais se inscrever nele. Da mesma forma, um curso

concluído não permite novos alunos. Já o cancelamento de cursos é uma alternativa à exclusão dos seus dados. Só podemos excluir um curso se não houver nenhum aluno inscrito nele. Assim, na operação de cancelamento, o sistema deve checar se há algum aluno inscrito. Se não houver, o curso pode ser excluído, mas, se houver, o curso deve ser registrado como cancelado.

## PROGRAMA PRINCIPAL

O programa principal agora já deve oferecer uma interface para o usuário, por meio da qual ele possa fazer inclusões, alterações, buscas e exclusões, para todas as entidades. A sugestão é que vocês ofereçam uma interface inicial semelhante a esta:

```
PresenteFácil 1.0
-----------------
> Início

[A] Meus dados
[B] Meus cursos
[C] Minhas inscrições

[S] Sair

Opção: _
```

Lembrem-se de que não colocamos código de interface com o usuário (visão) na mesma classe que o acesso aos dados (modelo). Tentaremos seguir o **padrão MVC (https://pt.wikipedia.org/wiki/MVC)**. Assim, vocês deveriam criar uma classe `VisaoCurso` que conteria todas as operações de entrada e de saída de dados relacionadas a cursos (não inclui o menu acima). Por exemplo, vocês poderiam ter pelo menos uma função `leCurso()` e outra `mostraCurso()`. Finalmente, teriam uma outra classe responsável pela lógica da operação que poderia se chamar `ControleCurso`. Essa última classe seria responsável pelo menu e pela lógica das operações de inclusão, alteração e exclusão, entre outras. Ela acessaria os arquivos necessários, bem como chamaria as funções da visão.

Neste TP1, também não faremos a opção C, das inscrições. Isso ficará para o TP2.

## CÓDIGO QUE JÁ ESTÁ PRONTO

Nesse projeto, vocês devem necessariamente usar o **CRUD genérico (https://github.com/kutova/AEDsIII/tree/main/CRUD2)** que desenvolvemos em sala como base. Nosso CRUD cria registros com a seguinte estrutura:

- Lápide - Byte que indica se o registro é válido ou se é um registro excluído;
- Indicador de tamanho do registro - Número inteiro (short) que indica o tamanho do vetor de bytes;

- Vetor de bytes - Bytes que descrevem a entidade (obtido por meio do método `toByteArray()` do próprio objeto da entidade).

Além disso, vocês precisarão usar as classes **TabelaHashExtensível (https://github.com/kutova/AEDsIII/tree/main/TabelaHashExtensivel)** e **Árvore B+ (https://github.com/kutova/AEDsIII/tree/main/ArvoreBMais)** que disponibizei para criar os índices. Não vale inventar uma nova estrutura de dados para os índices nesse projeto, ok?

## O QUE DEVE SER FEITO?

- Implementar o CRUD de Usuários.
- Implementar o CRUD de Cursos, assegurando que cada curso pertença a um usuário específico.
- Implementar o relacionamento 1:N com o par (idUsuario; idCurso) usando a Árvore B+.
- Criar a visão e o controle de usuários. Assegurar que um usuário não possa ser excluído se algum curso ativo estiver vinculada a ele. Se não, os cursos inativos devem ser removidos também.
- Criar a visão e o controle de cursos. Um novo curso deverá ser automaticamente vinculado ao usuário ativo no sistema.

Observe que para tudo funcionar, vocês precisarão acessar os arquivos e as visões de usuários e de cursos em todas as classes de controle.

## FORMA DE ENTREGA

- **Código** - Vocês devem postar o seu trabalho no GitHub **(https://replit.com/)** e enviar apenas o URL do seu projeto. Criem um repositório específico para este projeto (ao invés de mandar o repositório pessoal de algum de vocês em que estejam todos os seus códigos). Acrescentem um arquivo **readme.md** ao projeto que será o relatório do trabalho de vocês (explicado abaixo).
- **Relatório** - O relatório deve começar com a **lista dos participantes** do trabalho prático e, em seguida, ter uma **descrição completa** do que o sistema faz. Capturem algumas telas e citem os nomes das classes que foram criadas. Expliquem todas as operações especiais que foram implementadas. O objetivo é que vocês facilitem ao máximo a minha correção, de tal forma que eu possa entender com facilidade tudo aquilo que fizeram e dar uma nota justa. No relatório, vocês devem, necessariamente, responder ao seguinte ***checklist*** (copie as perguntas abaixo para o seu relatório e responda sim/não em frente a elas, justificando a resposta quando necessário):
    - Há um CRUD de usuários (que estende a classe ArquivoIndexado, acrescentando Tabelas Hash Extensíveis e Árvores B+ como índices diretos e indiretos conforme

necessidade) que funciona corretamente?
  - Há um CRUD de cursos (que estende a classe ArquivoIndexado, acrescentando Tabelas Hash Extensíveis e Árvores B+ como índices diretos e indiretos conforme necessidade) que funciona corretamente?
  - Os cursos estão vinculados aos usuários usando o idUsuario como chave estrangeira?
  - Há uma árvore B+ que registre o relacionamento 1:N entre usuários e cursos?
  - Há um CRUD de usuários (que estende a classe ArquivoIndexado, acrescentando Tabelas Hash Extensíveis e Árvores B+ como índices diretos e indiretos conforme necessidade)?
  - O trabalho compila corretamente?
  - O trabalho está completo e funcionando sem erros de execução?
  - O trabalho é original e não a cópia de um trabalho de outro grupo?
- **Vídeo de demonstração** - Grave um vídeo de **até 3 minutos** (captura de tela com narração em áudio) mostrando as principais operações do seu sistema. Se o vídeo ficar grande demais para o GitHub, vocês podem publicá-lo no YouTube e compartilhar o link ou usar a própria ferramenta do Canvas para captura de vídeo.

Lembre-se de que, para essa atividade, eu avaliarei tanto o esforço quanto o resultado. Portanto, escrevam o relatório e gravem o vídeo de forma que me ajude a observar o resultado.

**Atenção:** As respostas incorretas ao *checklist* prejudicaram consideravelmente a nota do grupo. Se vocês disserem que fizeram algo que não foi implementado, a nota final será reduzida em 50% por resposta incorreta (duas respostas incorretas significam a nota zero). Além disso, se vocês disserem que algo está funcionando corretamente, mas a operação não funcionar direito, a nota final será reduzida em 25% por resposta incorreta. Dessa forma, quando necessário, justifiquem as respostas ao *checklist*. A falta do relatório no repositório implicará em perda de 50% dos pontos obtidos na atividade. A falta do vídeo implicará, da mesma forma, em perda de 50% dos pontos. Se os dois faltarem, a nota será, automaticamente, zero.

## DISTRIBUIÇÃO DE PONTOS

Essa atividade vale 5 pontos. A rubrica de avaliação estabelece os critérios que serão usados na correção.

**Atenção**: o TP é específico por grupo. TPs iguais receberão a nota zero (independentemente de quem realmente fez o trabalho).

Se tiverem dúvidas sobre o trabalho a fazer, me avisem. Não deixem de observar que o URL

com o código no GitHub deve ser entregue até o dia especificado na atividade.